Young Thames LLP
📍 C-84, Sector 2, Noida, Uttar Pradesh
📞 +91 11 3585 0602 | +91 0120-4999484 🌐 www.youngthames.com | ✉️ hr@youngthames.com

Technical Knowledge Assessment – Intern Evaluation
(To be handwritten, scanned, and submitted to HR for evaluation)

General Instructions (Strictly to be Followed)

1. This is a **closed assessment** – no use of Google, ChatGPT, AI tools, textbooks, or online material. Answers must reflect **your own knowledge, skill, and reasoning**.
2. Use **only black pen** for writing answers.
3. Answers must be written on **plain A4 sheets**. If not available, you may use a notebook/register. Write neatly and clearly.
4. Each question must be answered with **elaboration, structured explanation, and examples** wherever applicable.
5. Diagrams, flowcharts, or pseudo-code may be used to clarify concepts.
6. Submissions must be **scanned clearly** (no blurred images) and emailed to **hr@youngthames.com** within the allotted time.

Section 1 – Self-Rating (Honesty Matters)
Rate yourself on a scale of **1 (Beginner) to 10 (Expert)** in the following technologies. Add remarks if you have built any projects, apps, or proof-of-concepts using them.

Technology	Rating (1–10)	Remarks (Projects / Use Cases Worked On)
JavaScript (ES6+)		
Node.js (Backend APIs)		
Next.js (SSR/ISR)		
Electron.js (Desktop Apps)		
Docker (Containers)		
Kubernetes (Orchestration)		
AWS Cloud (EC2, S3, Lambda)		
Supabase (DB + Auth)		
GitHub / GitLab (Version Control + CI/CD)		
Vercel (Frontend Deployment)		
Grafana & Prometheus (Monitoring)		
Zookeeper (Distributed Systems)		
AI Frameworks (TensorFlow, PyTorch, LangChain)		

Section 2 – Core Technical Questions (Write in Detail)

1. **JavaScript & Web**
   - Explain the **Document Object Model (DOM)** with a real example. How does JavaScript manipulate the DOM in dynamic web applications?
   - Differentiate between **event bubbling** and **event capturing**. Draw a diagram for clarity.
2. **Node.js**
   - Explain the role of the **`os` module**. Give 3 examples of how it can be applied in real-world systems.
   - Describe the **event loop mechanism** in Node.js. Why is it important for non-blocking I/O?
3. **Electron.js**
   - What is **Inter-Process Communication (IPC)** in Electron? Explain with an example.
   - Differentiate between **MSI** and **EXE** installers. Why might a company choose one over the other?
4. **Next.js**
   - Explain the difference between **SSR (Server-Side Rendering)**, **SSG (Static Site Generation)**, and **ISR (Incremental Static Regeneration)**. Provide a scenario where each is ideal.
   - What are **API routes** in Next.js? How are they different from using Express with Node.js?

Section 3 –Technologies

5. **Cloud & Databases**
   - What is **Supabase**? Compare it with Firebase, highlighting **advantages and limitations**.
   - Provide a step-by-step outline of integrating Supabase Authentication with a Next.js app.
6. **Containers & DevOps**
   - Explain the difference between **Docker Images** and **Docker Containers**.
   - What is a **Pod** in Kubernetes? How does it differ from a Node in a cluster?
   - Describe a real-world use case of **Kubernetes Horizontal Pod Autoscaler (HPA)**.
7. **CI/CD & Monitoring**
   - Define a **CI/CD pipeline**. Draw a flow diagram showing the stages from code commit to deployment.
   - Compare **Jenkins** with **GitHub Actions**. Which one is better for a startup environment and why?
   - How do **Grafana and Prometheus** integrate to monitor an application's performance?
8. **Distributed Systems & AI**
   - What is **Apache Zookeeper**? Explain its importance in leader election and distributed system management.
   - What is **Bolt (Slack AI SDK)** and how can it be applied in automating workflows?
   - List 3 AI frameworks other than ChatGPT. Write their major application areas.

Section 4 – Scenario-Based Problem Solving

9. You are tasked with building a **real-time collaboration platform** (similar to Google Docs).
   - Which stack (Node.js, Supabase, Docker, Kubernetes, AWS, WebSockets) would you use?
   - Draw an architecture diagram and explain each component.
10. A **Next.js application** deployed on Vercel is performing slowly. Write 5 possible reasons for the slowdown and explain how you would resolve each.
11. Your **CI/CD pipeline** fails frequently during deployment. How would you troubleshoot the following issues?
    - Docker build errors
    - Kubernetes cluster misconfiguration
    - GitHub Actions runner timeouts
12. You are setting up **observability** for a SaaS app. Explain how you would configure Prometheus alerts and Grafana dashboards to monitor latency, uptime, and error rates.

Section 5 – Case Study (Comprehensive Evaluation)

Problem Statement:
Young Thames LLP is building a **Case & Document Management SaaS** for clients. The stack is:

- **Frontend:** Next.js (with SSR/ISR)
- **Backend:** Node.js with REST APIs
- **Database:** Supabase (Realtime + Auth)
- **Deployment:** Vercel (Frontend) + AWS EC2 (Backend with Docker)
- **Orchestration:** Kubernetes (EKS)
- **Monitoring:** Prometheus + Grafana
- **CI/CD:** GitHub Actions

Task:
Explain step-by-step:

1. Draw the **system architecture diagram** (Frontend, Backend, Database, Orchestration, Monitoring).
2. Define how **Supabase authentication & schema** will be designed.
3. Outline a **CI/CD pipeline** from GitHub → Build → Test → Deploy.
4. Provide a **Dockerfile snippet** and Kubernetes YAML strategy.
5. Explain how monitoring dashboards and alerts will ensure uptime.
6. Suggest a **backup & recovery plan** for critical l

Final Declaration by Intern

I hereby declare that the answers given are based on my knowledge and practical understanding.

Signature: _____ Full Name: _____ Date: _____